NLP Study Notes: From Embeddings to Transformers

Unit 1: Word Representations

A computer cannot work with words directly, so every word must be turned into numbers. The simplest method is one-hot encoding. Each word gets a vector as long as the vocabulary, with a 1 in one position and 0 everywhere else. One-hot vectors are very long and they do not show any meaning: the vectors for 'king' and 'queen' are as far apart as the vectors for 'king' and 'banana'.

Word embeddings solve this problem. An embedding is a short, dense vector, usually 100 to 300 numbers long, that is learned from a large text corpus. Words used in similar contexts get similar vectors. Word2Vec, introduced by Mikolov and others at Google in 2013, learns embeddings with two methods: CBOW predicts a word from its neighbours, and Skip-gram predicts the neighbours from the word. GloVe, from Stanford in 2014, learns embeddings from global word co-occurrence counts.

The main limit of Word2Vec and GloVe is that each word has only one vector. The word 'bank' gets the same vector in 'river bank' and 'bank loan'. These are called static embeddings. Contextual embeddings, produced by models such as BERT, give a different vector to the same word in different sentences.

Unit 2: Recurrent Networks and Their Limits

A Recurrent Neural Network (RNN) reads a sentence one word at a time and keeps a hidden state that carries information forward. This makes RNNs a natural fit for sequences. However, RNNs suffer from the vanishing gradient problem: when the gradient is passed back through many time steps, it becomes very small, so the network forgets information from early words.

LSTM (Long Short-Term Memory) networks reduce this problem using three gates: the forget gate, the input gate and the output gate. The gates decide what to keep and what to throw away in a separate cell state. GRU (Gated Recurrent Unit) is a simpler version with two gates: the update gate and the reset gate.

Even LSTMs have two weak points. First, they process words one after another, so training cannot be done in parallel across the sentence, which makes training slow on GPUs. Second, in an encoder-decoder model for translation, the whole input sentence is squeezed into one fixed-size vector. For long sentences, this single vector becomes a bottleneck and quality drops.

Unit 3: The Attention Mechanism

Attention was introduced for machine translation by Bahdanau, Cho and Bengio in 2014. Instead of using one fixed vector, the decoder looks back at all encoder hidden states at every step and decides how much weight to give each input word. These weights are called attention weights, and they always add up to 1 because they come from a softmax.

Self-attention applies the same idea inside one sentence: every word looks at every other word in the same sentence. Each word is turned into three vectors: a Query (Q), a Key (K) and a Value (V). The Query of one word is compared with the Keys of all words to get scores, and the output is a weighted sum of the Values.

Scaled dot-product attention is computed as softmax(Q K^T / sqrt(d_k)) V, where d_k is the size of the key vectors. The division by sqrt(d_k) is needed because dot products grow large when vectors are long; large values push the softmax into regions where gradients are very small. Scaling keeps training stable.

Multi-head attention runs several attention operations in parallel, each with its own learned Q, K and V projections. Different heads can learn different relations, for example one head may follow grammar links while another follows meaning. The outputs of all heads are joined and passed through a final linear layer. The original Transformer used 8 heads.

Unit 4: The Transformer

The Transformer was proposed in the 2017 paper 'Attention Is All You Need' by Vaswani and others at Google. It removes recurrence completely and uses only attention and feed-forward layers. Because there is no recurrence, all words in a sentence can be processed in parallel, which makes training much faster.

Since self-attention has no sense of word order by itself, the Transformer adds positional encodings to the input embeddings. The original paper used fixed sine and cosine functions of different frequencies. Many later models, such as BERT, learn the position embeddings instead.

The original Transformer has an encoder stack and a decoder stack, each with 6 layers. Every encoder layer has a multi-head self-attention block and a feed-forward block. Each block is wrapped with a residual connection followed by layer normalisation. The decoder adds a masked self-attention block, so a word cannot see future words, and a cross-attention block that looks at the encoder output.

The main cost of self-attention is that it compares every word with every other word, so time and memory grow with the square of the sequence length, written as O(n^2). This is why long documents are hard for standard Transformers.

Unit 5: BERT, GPT and Tokenization

BERT (Bidirectional Encoder Representations from Transformers) was released by Google in 2018. It uses only the encoder stack. BERT is pre-trained with two tasks: Masked Language Modelling, where about 15 percent of input tokens are hidden and the model predicts them, and Next Sentence Prediction. Because it reads the whole sentence at once, BERT is bidirectional and is good for understanding tasks such as classification and named entity recognition.

GPT (Generative Pre-trained Transformer), from OpenAI, uses only the decoder stack with masked self-attention. It is trained to predict the next token, reading from left to right. This makes GPT models autoregressive and well suited to text generation.

Modern models do not split text into whole words. They use subword tokenization. Byte Pair Encoding (BPE), used by GPT models, starts from single characters and repeatedly merges the most frequent pair into a new token. WordPiece, used by BERT, works in a similar way but chooses merges that most increase the likelihood of the training data. Subword methods can handle rare and new words by breaking them into known pieces, so there are almost no out-of-vocabulary words.

Retrieval-Augmented Generation (RAG) combines a retriever with a generator. The retriever finds passages related to the question from a document collection, and the language model writes the answer using those passages. RAG reduces hallucination and lets the model use documents it was never trained on.